Size Change Termination Condition for Injective Relations

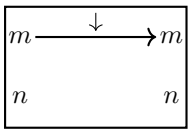
Stefano Berardi

April, 2025

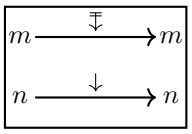
1 Introduction

Size change termination, originally introduced in [LJBA01], is a method used to prove the termination of recursive functions. The main idea behind size change termination lies in Fermat's proof principle of infinite descent: To proof a proposition P, assume P is false and reach a contradiction by constructing an infinite descending chain in a well-order. This principle can be applied in the context of recursive functions as follows: if the parameters of a function belong to a well-order, we can assume the function execution does not terminates, and derive a contradiction by forming an infinite descending chain with the values of the parameters. These infinite descending chains are formed by analyzing the relationships between the parameters of the function in each recursive call. Each relationship is captured by a structure known as the *size change graph*, which provides an abstract way to define how the parameters change during program execution. To understand the general idea behind this method, lets consider the following example of the Ackermann function, taken from [Lee09]:

```
a(m,n) = if m=0 then n+1 else
         if n=0 then 1a(m-1,1) else
         2a(m-1,3a(m,n-1))
```



G_1, G_2



G_3

Figure 1: Definition of the Ackermann function a and size change graphs G_1, G_2 and G_3

For each recursive call, labeled with 1,2 and 3, we can respectively define the size change graphs G_1, G_2 and G_3, where the edges $x \xrightarrow{\gamma} y$ represent the relationship between the values of the parameters initially passed to the function, and the values after each recursive call. When $\gamma = \downarrow$, it specifies that the value of x is strictly less than the value of y, and when $\gamma = \overset{=}{\downarrow}$, it specifies that the value of x is less than or equal to the value of y. Using these graphs we can then analyze the function's execution: assume a hypothetical infinite execution of the program. This will induce an infinite sequence of recursive calls $c = c_1c_2c_3...$ where every $c_i \in \{1, 2, 3\}$, which also induces an infinite sequence of size change graphs $G_{c_1}G_{c_2}G_{c_3}...$ It is then possible to compose these graphs, to form infinite paths using the relations between each one of the variables. For example, suppose we get an infinite execution $(13)^\omega$, i.e. the recursive call 1 and 3 are used infinitely one after the other. This induces the sequences of graphs $(G_1G_3)^\omega$ and the infinite path $m \xrightarrow{\downarrow} m \xrightarrow{\overset{=}{\downarrow}} m \xrightarrow{\downarrow} m...$ over the variable m, which progresses infinitely often and implies that this infinite execution is impossible.

1. Continue with Cyclic proofs (cite ikebuchi) and use an example to explain the intuition why GTC is STC.
2. Explain how this can be abstracted in a more general way for any circular system with progressing condition
3. Explain why in general circular proof systems satisfy injectivity

4. state what is the objective of the paper

References

[Lee09] Chin Soon Lee. Ranking functions for size-change termination. *ACM Trans. Program. Lang. Syst.*, 31(3), April 2009.

[LJBA01] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '01, page 81–92, New York, NY, USA, 2001. Association for Computing Machinery.